Call Transcript LLM Design

LLM call design for CA call summarisation, CIT extraction and topic flagging (GPT-5.1, DSPy + GEPA, Durable Functions)

Oct 30, 2026 · @Viswanath Harith

Executive summary

Run each call through one Durable Functions orchestration in three steps:

1. One sequential **conversation map** call. It builds a hierarchical index of the call and warms the transcript cache.
2. A parallel fan-out for the summary, CIT detection and per-domain topic flagging.
3. A second fan-out for CIT parameter extraction.

All calls use GPT-5.1, with reasoning effort set per stage. DSPy 3.3 with GEPA tunes the prompts offline; production runs frozen, versioned prompts loaded from Postgres.

Decision	Recommendation	Why
Orchestration	Durable Functions: warm-up call, then parallel stages	Parallel calls sent at the same instant all miss the cache; the map call is useful work that warms it first
Prompt layout	Transcript first, stage instructions last; identical model settings within a cache family	Caching matches exact prefixes only, and some request settings change the prefix
Summary	Structured JSON, guideline-driven, every bullet cites turn IDs	Grounded, auditable and editable section by section
CIT	Detect once over a compact catalog, then extract once per candidate	A catalog-driven, generic output envelope, validated in code
Topics (350+)	Hierarchical routing to domains, then per-domain adjudication; lexicon and optional vector safety nets	Similarity is not relevance; picking from hundreds of labels at once misses rare ones
Retrieval	No chunking or vector RAG over the transcript; the conversation map is the vectorless index	A 12k-token transcript fits easily in the context window; chunking loses corrections and back-references

Decision	Recommendation	Why
Summary editing	Synchronous API call: preset action catalog, section-scoped, sources found via the map	Fast (1–8 s) and cheap; validated against the transcript
Optimization	DSPy + GEPA offline only; export the optimized instructions	Bank-grade auditability, and full control of prompt layout and caching
Cost	About $0.22 per call, roughly $2.4k/month at 500 calls/day	Caching saves about 40%; the bill is small next to the CA time saved

Biggest open technical risk. OpenAI's caching docs list structured-output format and reasoning effort among the settings that change the cached prefix. Before relying on caching *across* stages, run a one-day spike on Azure GPT-5.1 that measures `cached_tokens`. Caching *within* a stage's fan-out is safe by design.

Scope, assumptions and design principles

This document covers application logic only: LLM call design, prompts, catalogs, orchestration logic, editing and optimization. Hosting, networking and data-residency setup are out of scope; they are covered in the infra design.

Assumptions

Item	Assumption
Volume	~500 calls/day, 150 CAs, 45-minute average call
Transcript size	~9–12k tokens per call; German, French and Italian tokenize 20–40% heavier than English
Input	Diarized transcript with speaker roles CA/CLIENT. Client data reaches the model as spoken; only identifier types no task needs (date of birth, AHV, card numbers) are replaced by placeholders, configurable per type
Models	Azure OpenAI `gpt-5.1` (Data Zone Standard), pinned version, auto-upgrade off
Catalogs	CIT task catalog and Topics catalog (350+) in PostgreSQL, versioned

Item	Assumption
Latency target	Results in the CA review UI within ~1–2 minutes of the transcript arriving

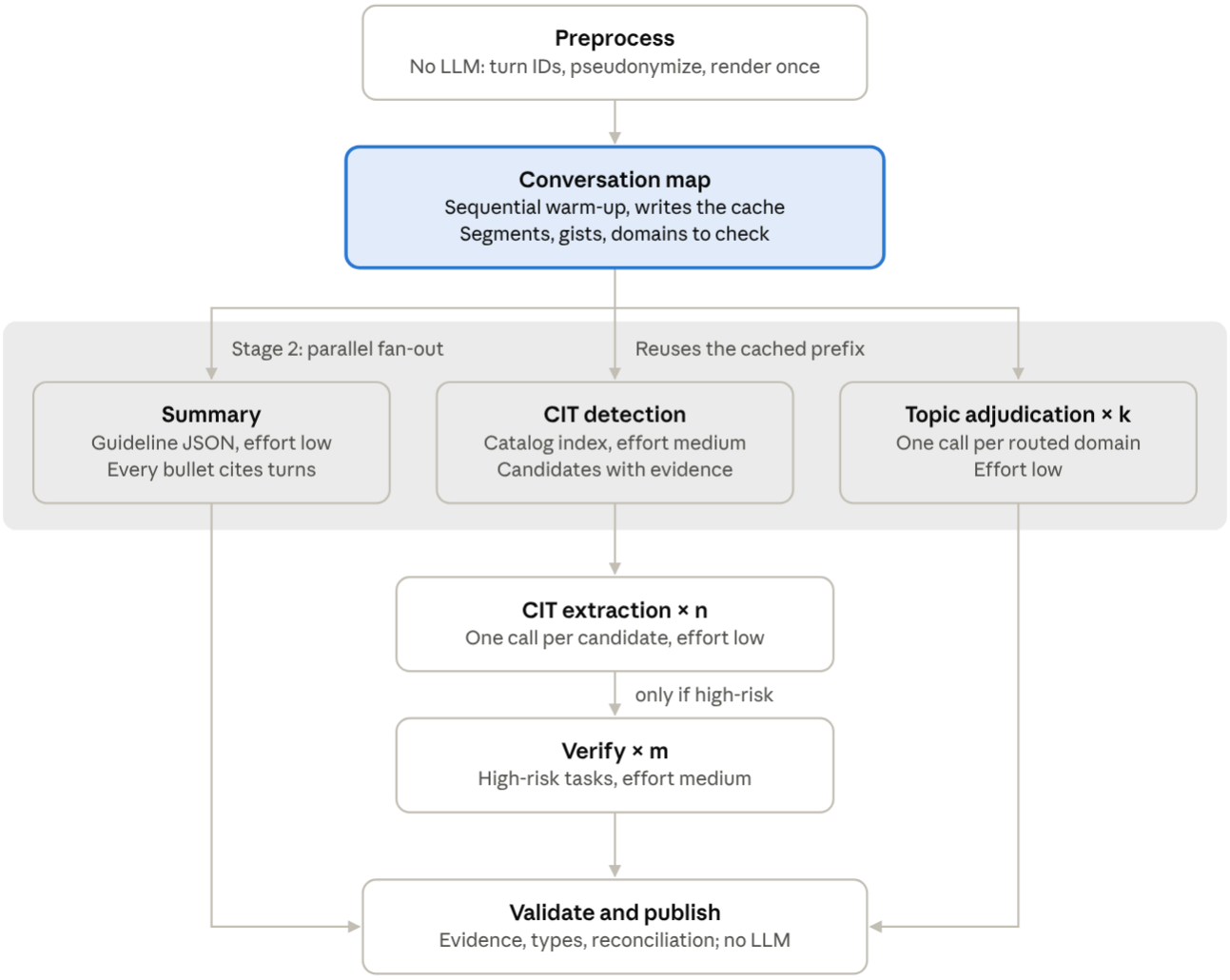
Design principles

1. **Long context, not RAG, over the transcript.** Every analysis call sees the whole call, and retrieval is used only to choose which catalog entries to show.
2. **Evidence or it didn't happen.** Every summary bullet, CIT parameter and topic flag carries turn IDs and a verbatim quote, and code checks the quote exists in the transcript.
3. **Catalogs are data, prompts are code.** Task and topic definitions live in Postgres and are injected at runtime. Instruction text is versioned, approved and frozen.
4. **Recall over precision for regulatory items.** A false positive costs the CA one click; a missed consent or AML signal costs far more.
5. **Design for the cache, but never depend on it.** Prompt layout maximizes prefix reuse, while correctness never relies on a cache hit.
6. **Human in the loop for anything legally binding.** Consent, waivers and contract actions are always confirmed by the CA before they reach downstream systems.

End-to-end pipeline and Durable Functions orchestration

Yes, both use cases fit one Durable Functions orchestration with parallel activities. On caching, the answer is neither fully parallel nor fully sequential: send one sequential warm-up call, then fan out.

One warm-up call, then parallel fan-out on the cached transcript



Pipeline orchestration · 1 warm-up call, 2 parallel fan-outs

Only the map call runs alone. Every later call reuses the transcript prefix it cached, and a failure in one branch publishes the run as partial instead of failing it.

The Durable fan-out/fan-in pattern schedules activities concurrently, waits for the whole group, and returns results in task order (Microsoft Learn).

#	Activity	Waits for	Runs in parallel	Model / reasoning effort	Cache family
0	`preprocess` (no LLM)	—	—	—	—
1	`build_conversation_map`	0	No (warm-up)	gpt-5.1 / low	T (writes)
2a	`summarize`	1	Yes	gpt-5.1 / low	T
2b	`detect_cits`	1	Yes	gpt-5.1 / medium	T*
2c	`adjudicate_topics` × k domains	1	Yes	gpt-5.1 / low	T
3	`extract_cit` × n candidates	2b	Yes	gpt-5.1 / low	T

#	Activity	Waits for	Runs in parallel	Model / reasoning effort	Cache family
4	`verify_cit` × m high-risk	3	Yes	gpt-5.1 / medium	T*
5	`validate_and_publish` (no LLM)	all	—	—	—

T = shared transcript prefix. T* = may land in a separate cache entry because its reasoning effort differs (see the next section).

Why warm up first, then fan out

- **Simultaneous requests all miss.** A cache entry exists only after a request has processed the prefix, so calls sent at the same instant on a cold transcript all pay full price.
- **The map call is real work.** Stages 2–3 consume it anyway, so using it as the warm-up adds no wasted call. The added latency is one call of ~15–25 s.
- **Route with a cache key.** On models before GPT-5.6, OpenAI recommends a stable `prompt_cache_key` to route related requests to the same cache, keeping each key around 15 requests per minute (OpenAI). Use `prompt_cache_key = <transcript_id>`: one call produces roughly 8–12 requests, well under that limit.
- **The in-memory cache is short-lived.** It typically survives 5–10 minutes of inactivity, up to an hour (OpenAI). The whole pipeline finishes within about 2 minutes, so this is no constraint.

Orchestrator (Python v2 programming model)

```
@app.orchestration_trigger(context_name="ctx")
def process_call(ctx: df.DurableOrchestrationContext):
    run = ctx.get_input()  # {"run_id", "transcript_id"}: IDs only, never
content
    retry = df.RetryOptions(first_retry_interval_in_milliseconds=5000,
                            max_number_of_attempts=4)
    act = lambda name, payload: ctx.call_activity_with_retry(name, retry,
payload)

    yield act("preprocess", run)
    cmap = yield act("build_conversation_map", run)          # warms the
transcript cache

    stage2 = [act("summarize", run), act("detect_cits", run)]
    stage2 += [act("adjudicate_topics", {**run, "domain": d})
               for d in cmap["domains_to_check"]]           # k domains,
capped at 8
    res = yield ctx.task_all(stage2)
```

```
    cands = res[1]["candidate_ids"]
    yield ctx.task_all([act("extract_cit", {**run, "candidate_id": c}) for c
in cands])

    risky = yield act("select_high_risk", run)
    if risky:
        yield ctx.task_all([act("verify_cit", {**run, "candidate_id": c}) for
c in risky])

    yield act("validate_and_publish", run)
```

Durable rules for this workload

- **Pass IDs, not transcripts.** Activity inputs and outputs are written into the orchestration history and reloaded on every replay. Microsoft's guidance is to write large results to an external store and return a reference (Microsoft Learn). For a bank this matters twice over: history in task-hub storage would otherwise hold client data.
- **Activities load and persist their own data.** Each activity reads the canonical transcript from Postgres or Blob, writes its result to Postgres, and returns a small status object.
- **Make activities idempotent.** Key each result on `(run_id, stage, item_id, prompt_version_id)`. If the row already exists, return it, because Durable delivers at least once.
- **Fail soft per item.** `task_all` fails as soon as any task fails. Activities should catch LLM errors, record `status = failed`, and return normally. The run is then published as `partial`, and the UI shows what is missing.
- **Keep the orchestrator deterministic.** No I/O, clock reads or randomness in its body; anything like that goes in an activity.
- **Throttle in two layers.** Set `maxConcurrentActivityFunctions` in `host.json`, and honour `Retry-After` on HTTP 429 inside the activity before Durable's own retry kicks in.
- **Interactive summary edits do not go through Durable.** They are synchronous API calls (see the editing section).

Prompt layout and prefix caching strategy

Every analysis call on a transcript uses the same three-part layout. The warm-up only pays off if the request settings are identical too, so the design also pins those.

```
[tools]      OUTPUT TOOLS: all six output contracts, strict JSON schemas,
fixed order
[developer]  GLOBAL_RULES: grounding, citation, masking/normalisation and
language rules (~900 tokens)
[user]       <transcript id="..." lang="de"> [T001][CA] ... [T612][CLIENT]
... </transcript>
------------ shared, cacheable prefix ends here (~13–16k tokens) ------------
[user]       STAGE BLOCK: frozen stage instructions (versioned)
             + stage inputs (guidelines / catalog slice / task definition /
map extract)
             + "Respond by calling <tool_name>."
```

The catch: request settings are part of the prefix

OpenAI's caching guide lists settings that change the cached prefix even when the message text is identical (OpenAI):

- `model`
- `tools`
- `text.format`, i.e. the Structured Outputs schema
- `reasoning.effort`
- `text.verbosity`

If each stage sent its own `response_format` schema, the map call would warm a prefix that no other stage can reuse.

Recommended: one uniform envelope for the T family

Setting	Value for every T-family call	Why
Output contract	The same list of six strict tools in every call (`emit_conversation_map`, `emit_summary`, `emit_cit_candidates`, `emit_cit_extraction`, `emit_topic_flags`, `emit_verification`)	The tool list is identical, so the prefix is identical
Selecting the output	`tool_choice` forces the one tool the stage needs	OpenAI's guidance for stable caching is to keep tool definitions fixed and restrict *which* tool is used, not to change the list (OpenAI)

Setting	Value for every T-family call	Why
`text.format` / `verbosity`	Not used	Output length comes from the guidelines and schema caps
`reasoning.effort`	`low`	Detection and verification at `medium` are accepted as separate cache entries; test whether `low` is good enough on the golden set
`prompt_cache_key`	`transcript_id`	Routing hint for pre-5.6 models; confirm in the spike that the Azure deployment accepts it, and drop it if rejected

Generic extraction contract. `emit_cit_extraction` is one generic schema: `{task_code, instance_id, params: [{param_code, value, status, evidence[]}]}`. Per-task typing (email, IBAN, enum, tri-state) is enforced in code against the catalog. This keeps the tool list stable while the catalog changes. Generic closed sets (statuses, tri-states, value kinds) are still enforced as enums while the model generates; catalog-specific enums and regex formats are validated in code (see Implementation details).

Validate before relying on it (day-one spike, about 1 day)

1. Replay 20 transcripts through the pipeline with the uniform envelope, and again with per-stage `text.format`.
2. Log `usage.prompt_tokens_details.cached_tokens` per stage. On pre-5.6 models it is reported rounded down to multiples of 128 (OpenAI).
3. **Proposed target:** at least 70% of input tokens cached on stage 2–4 calls.
4. **Watch Data Zone routing.** Caches are not reused across regional processing boundaries (OpenAI), so Data Zone routing may lower the hit rate.

If hits stay poor, keep the quality-first layout anyway. The cost of no caching is about $1.7k/month (see the cost section), which is not worth bending prompts or schemas for.

Alternative considered: catalog-first prefix. Putting a static catalog before the transcript caches the catalog across all calls of the day. That only wins when the catalog is larger than the transcript, e.g. the single-shot 350-topic baseline at ~25k tokens. The recommended hierarchical topic design needs only small catalog slices, so a transcript-first prefix wins.

Application-side caching and measuring cached tokens

The application controls caching layers that are more reliable than the provider's prefix cache. The three that matter most:

1. a content-addressed store of stage results;
2. dependency-aware incremental reprocessing;
3. deterministic prompt rendering.

The provider cache then becomes a bonus on top. Semantic caching is deliberately excluded.

Caching layers the application controls

Layer	What is cached	Key	What it saves	When
1. Stage result store	Every stage output (map, summary, candidates, extractions, flags)	`sha256(canonical_transcript)` + stage + item + prompt version + catalog version + deployment + settings hash	100% of the call on retries, replays, UI reloads and re-runs of unchanged stages	MVP; it doubles as the idempotency key
2. Incremental reprocessing	The artifact graph: map → summary / CIT / topics	Catalog diff → affected task or topic codes → affected domains	A change to one AML topic re-runs only AML adjudication, reusing the stored map	Phase 1.5
3. Rendered prompt cache	Catalog renderings (detection index, per-domain topic blocks, guidelines) and tool schemas	Catalog version + renderer version	CPU, and above all a byte-identical prefix on every call	MVP (in-process LRU)
4. Edit result cache	Edit outputs	`(summary_version_id, action_code, params)`	Repeated translate or CRM-format requests	MVP (cheap)
5. Embedding cache	Topic exemplar and segment embeddings	Catalog version / segment hash	Embedding calls	Phase 2, with the vector net

Layer	What is cached	Key	What it saves	When
6. Stage short-circuits	— (skips calls deterministically)	—	No candidates → no extraction; no high-risk task → no verification; trivial calls (voicemail, under N turns) → reduced pipeline	MVP

Incremental reprocessing in practice. Every `topic_flag` and `cit_instance` row records the catalog version and domain it was judged under.

1. When catalog version N+1 is approved, diff it against N to get the changed codes and their domains.
2. Re-run only `adjudicate_topics(domain)` or `extract_cit(task)` for calls in the agreed scope (for example the last 90 days), against the stored transcript and map.
3. Cost is roughly calls × affected domains × $0.016, instead of a full re-run.

Deliberately not used

- **Semantic (similarity) response caching**, e.g. a gateway semantic cache. Every transcript is unique, and returning a "similar" call's result would be both wrong and a data-leakage risk between clients.
- **Provider-side conversation state** (Responses API `previous_response_id` with `store=true`). Keep every call stateless, so that nothing about a client persists at the provider beyond the request.
- **Extended 24-hour prompt-cache retention.** It would keep client data in cache state for up to 24 hours, and the pipeline finishes in about 2 minutes, so it is not needed here.

Client data in application stores. Because prompts and outputs carry client data, the stage result store, conversation segments, summary versions, extracted values and any recorded replay fixtures are client data too. Encrypt them at rest, limit access to the CA's own clients and to support roles, and follow the transcript's retention. Telemetry logs token counts, latencies and prefix hashes only, never prompts or completions.

Measuring cached tokens on Azure GPT-5.1

Where the number comes from. Cache hits are reported as `cached_tokens` under `usage.prompt_tokens_details` in the Chat Completions response (Microsoft Learn). In the Responses API the field is `usage.input_tokens_details.cached_tokens`.

How hits are counted.

- Caching starts after the first 1,024 identical tokens and grows in steps of 128 tokens.
- One differing character within those first 1,024 tokens yields `cached_tokens = 0` (Microsoft Learn).

Log it yourself, per call. In early 2025, the API Management token-metric policy recorded only total, prompt and completion tokens, with cached-token support on the roadmap (Microsoft Q&A). So log from the `usage` object into `llm_call`, and emit the same values as OpenTelemetry span attributes or Application Insights custom metrics with dimensions `stage`, `prompt_version` and `deployment`.

Also log `prefix_sha256`, a hash of the rendered tools + developer + transcript messages. It separates the two failure causes:

- **Same hash, `cached_tokens = 0`:** a timing or routing miss.
- **Different hashes:** a rendering bug that broke byte identity.

```
u = resp.usage
record_llm_call(
    stage=stage, item_id=item_id, prompt_version_id=pv.id,
deployment=deployment,
    input_tokens=u.prompt_tokens,
    cached_tokens=(u.prompt_tokens_details.cached_tokens if
u.prompt_tokens_details else 0),
    output_tokens=u.completion_tokens,
    reasoning_tokens=(u.completion_tokens_details.reasoning_tokens if
u.completion_tokens_details else 0),
    prefix_sha256=prefix_hash, prompt_cache_key=transcript_id,
latency_ms=elapsed_ms)
```

```
-- daily cache hit rate per stage (proposed target >= 0.70 for stages 2-4;
the map call is expected to be ~0)
select date_trunc('day', created_at) as day, stage,
       sum(cached_tokens)::numeric / nullif(sum(input_tokens), 0) as
cache_hit_rate,
       count(distinct prefix_sha256) filter (where stage <>
'build_conversation_map') as distinct_prefixes
from llm_call
group by 1, 2 order by 1 desc, 2;
```

Stage 0: preprocessing and transcript canonicalization

Stage 0 is deterministic code. It produces one canonical transcript string (masked types replaced) per call, and every LLM call then sends exactly those bytes.

1. **Turn segmentation.** Merge speech-recognition fragments into speaker turns, each with an ID (`T001` …), a role (`CA` / `CLIENT` / `OTHER`) and start/end timestamps.
2. **Detection and configurable masking.** Detect every client identifier, then replace only the types listed in `PSEUDONYMIZE_TYPES` with stable placeholders (`[DOB_1]`, `[AHV_1]`). All other identifiers stay as spoken, so extraction gets real values. The mapping and a detection log stay in Postgres.
   - Spoken forms need custom recognizers even when nothing is masked: the detection log and the second reading of IBANs and emails depend on them ("m punkt müller at bluewin punkt ch", IBANs read out digit by digit).
   - A masked entity gets the same placeholder throughout the call.
3. **Light normalization only.** Keep corrections and hesitations ("nein, doch nicht", "actually, cancel that"). They are evidence for whether a task is cancelled.
4. **Render once, hash, store.** Persist `canonical_text`, `sha256`, `token_count` and `dominant_language`. Activities load this string and never re-render it, because any whitespace difference breaks the cache prefix.
5. **Quality flags.** Record low diarization confidence, a high share of unknown speakers, or speech-recognition confidence below a threshold. A flagged call still runs, but the map call is told to infer speaker roles and the UI shows a warning.

A 45-minute call is ~9–12k tokens and even a 3-hour call stays far below the context window, so no length-based splitting is needed.

Pseudonymization and data minimisation

Client data may reach the model, so this layer no longer hides everything. It detects every identifier, sends what extraction needs as spoken, and masks only identifier types that no task needs. One setting, `PSEUDONYMIZE_TYPES`, decides which types are masked.

`PSEUDONYMIZE_TYPES`	The model receives	When
`DOB,AHV,CARD` (recommended)	Real names, emails, IBANs, phone numbers and addresses; date of birth, AHV and card numbers masked because no task needs them	Default for this application
`none`	The transcript as spoken	No minimisation wanted

`PSEUDONYMIZE_TYPES`	The model receives	When
`all`	Placeholders only; real values re-hydrated locally	A deployment not approved for client data

Pipeline

1. **Normalize spoken forms.** Work on a shadow copy with an offset map back to the original text.
   - Collapse spelled-out letters ("M wie Martha", Swiss and NATO spelling alphabets).
   - Turn digit words into digits in DE/FR/IT/EN ("neun drei" → 93, "quatre-vingt-dix" → 90).
   - Map "at / ät / arobase / chiocciola" → `@`, "punkt / point / punto / dot" → `.`, and "Bindestrich / tiret / trattino" → `-`.
   - Turn on inverse text normalization in speech-to-text where available, so less of this is needed.
2. **Match known entities first.** This gives the best precision and recall. Load this client's own CRM attributes for the call: names, email addresses, phone numbers, IBANs, account and portfolio numbers, address and date of birth. Match them exactly and fuzzily: edit distance, phonetic matching (Kölner Phonetik or Double Metaphone), and variants ("Herr Müller", "Mueller", "Hans"). Mask the CA's name the same way.
3. **Pattern recognizers.**
   - IBAN, with an ISO 13616 mod-97 check; CH/LI IBANs are 21 characters.
   - Swiss AHV/AVS number (`756.XXXX.XXXX.XX`).
   - Phone numbers (+41 / 0xx).
   - Email addresses.
   - Card numbers (Luhn check).
   - The bank's internal account and portfolio formats.
   - Date of birth when the context says so ("geboren am", "né le").
   - Swiss postal code plus town.
4. **Multilingual NER for whatever is left** (names of third parties, organizations, places): Microsoft Presidio (MIT) with spaCy DE/FR/IT/EN pipelines, or an in-house transformer NER model. Presidio also hosts the custom recognizers from steps 2–3, so everything runs in one engine.
5. **Resolve overlapping detections.** Priority is known entity, then pattern, then NER. Thresholds for person names favour recall.

6. **Assign placeholders for masked types.** They are typed and stable per transcript: the same normalized value always gets the same token (`[DOB_1]` , or `[PERSON_2]` when names are masked). Name variants are linked to one person. Never put inferred roles inside the token.
7. **Send everything else as spoken.** Unmasked identifiers stay exactly as transcribed. The model normalizes them on extraction and code checks the result (see the CIT section). Amounts, currencies, product names and countries were never masked.

Storage, re-hydration and QA

- **Detection log.** `pii_detection(transcript_id, turn, entity_type, surface, normalized_value, masked)` records every identifier found and whether it reached the model as a placeholder or as the real value. Reviewers and auditors see exactly what the model saw.
- **Mapping table, masked types only.** `pseudonym_map(transcript_id, placeholder, entity_type, value_encrypted, source)` , encrypted with a customer-managed key. It follows the transcript's retention.
- **Re-hydration** of masked types happens server-side at UI render and CRM write, never in a prompt.
- **Egress guard on masked types.** A raw date of birth, AHV or card number in a prompt blocks the call. Unmasked types pass by design.
- **Detection quality.** The second reading of spoken values depends on detection, so measure recall per type on the golden set. **Proposed gate:** ⩾ 99% for masked types.

```
from presidio_analyzer import AnalyzerEngine, Pattern, PatternRecognizer

MASK = mask_types(settings.pseudonymize_types)          # e.g. {"DOB", "AHV",
"CARD"}

ahv = PatternRecognizer(supported_entity="CH_AHV",
        patterns=[Pattern("ahv", r"\b756[.\s]?\d{4}[.\s]?\d{4}[.\s]?\d{2}\b",
0.7)])
analyzer = AnalyzerEngine(nlp_engine=multilingual_spacy_engine(),   #
de/fr/it/en models
                          supported_languages=["de", "fr", "it", "en"])
analyzer.registry.add_recognizer(ahv)

def pseudonymize(text: str, lang: str, known: KnownEntities) -> tuple[str,
PlaceholderMap, list[Detection]]:
    norm, offsets = normalize_spoken_forms(text, lang)
    spans = known.find(norm) + analyzer.analyze(norm, language=lang)
    spans = resolve_overlaps(spans, priority=("KNOWN", "PATTERN", "NER"))
    pmap, out, log = PlaceholderMap(), text, []
```

```
    for s in sorted(spans, key=lambda s: s.start, reverse=True):
        a, b = offsets.to_original(s.start, s.end)
        value, masked = canonical(norm[s.start:s.end]), s.entity_type in MASK
        log.append(Detection(s.entity_type, text[a:b], value, masked))   #
what the model saw, per identifier
        if masked:
            out = out[:a] + pmap.token_for(s.entity_type, value) + out[b:]
    return out, pmap, log
```

Stage 1: conversation map (hierarchical structure)

The conversation map is a hierarchical table of contents of the call, generated by one LLM call. It is the vectorless index that summary, topic routing, editing and the UI all build on.

Approach: an in-house tree index, navigated by reasoning. The pipeline does not chunk the transcript and search embeddings. It builds a small two-level outline of the call once: phases, then segments, each with a turn range, title and gist. Later steps find content by reading that outline, first with Postgres full-text search and, only if that is ambiguous, with one small LLM call over the outline. Everything runs in the bank's own code, on its Azure OpenAI deployment and PostgreSQL, with no external retrieval library or service.

Why build it in-house. A call has a simple structure: two levels and contiguous turn ranges. One LLM call with a strict schema captures it, and roughly 300 lines of Python plus one table store and query it. A generic document-indexing framework adds nothing here. The builder, validator, fallback and navigator are specified below, under *Implementation*.

Output contract (`emit_conversation_map`)

```
{
  "call_purpose": "Annual review; client wants to restructure mandate and set
up e-banking for spouse",
  "segments": [
    {
      "segment_id": "S04", "phase": "needs_and_situation",
      "turn_start": "T041", "turn_end": "T088",
      "title": "Inheritance and source of funds",
      "gist": "Client expects an inheritance; CA asks about origin and
timing.",
      "initiated_by": "CLIENT",
      "domains": ["AML", "INVESTMENT_ADVICE"],
      "signals": ["instruction", "consent_discussion"]
    }
  ]
}
```

The validator derives the phase level (`P1…Pn`, one per run of consecutive segments with the same phase) and `domains_to_check` (the union of segment domains plus lexicon hits). The model does not output them.

Rules in the stage instructions

- **Two levels.** Phases (opening, needs and situation, advice, administration, closing) contain topical segments. A 45-minute call typically has 6–20 segments.
- **Full coverage.** Segments are contiguous and cover every turn. Code checks for gaps and overlaps.
- **Over-include domains.** Codes come from the ~15–25 L1 domains of the topic catalog (see the topics section), and the instruction is to include a domain when in doubt. A wrong extra domain costs one cheap adjudication call; a missed one loses topics.
- **Signals are a closed set.** `instruction`, `consent_discussion`, `complaint`, `product_mention`, `risk_disclosure`, `cancellation`. They are hints, not decisions.

Who uses the map

Consumer	How it uses the map
Summary	Section skeleton and ordering; each summary bullet stays inside its segment's turn range
CIT detection	Segment signals are shown as hints; detection still reads the full transcript
Topic flagging	`domains_to_check`, plus lexicon hits, decides which per-domain adjudication calls run

Consumer	How it uses the map
Summary editing	Vectorless retrieval: section → segment → turn range, so an edit sends 1–4k tokens instead of the whole call
CA UI	A clickable call outline with jump-to-audio by turn timestamp

Settings. gpt-5.1, reasoning effort `low`, ~1.5k output tokens, about 15–25 s. Keep the map a separate call from the summary rather than merged into it. The map is structural and deliberately over-inclusive, while the summary is curated for the CA, and merging them blurs both.

Implementation: builder, validator, fallback and navigator

The conversation map takes five pieces of in-house code. Only the builder and, when needed, the navigator call the LLM.

#	Component	Kind	What it does	Cost / latency
1	`build_conversation_map`	LLM call (gpt-5.1, effort low, forced tool)	Transcript → outline	~$0.05, 15–25 s
2	`repair_map`	Deterministic Python	Guarantees full, contiguous coverage; fixes small defects; derives phases and `domains_to_check`	< 10 ms
3	`fallback_map`	Deterministic Python	Used if the map is still invalid after one retry	< 10 ms
4	`conversation_segment` + full-text index	PostgreSQL	Stores the outline; makes titles and gists searchable	< 10 ms
5	`locate_segments`	Full-text search, then an optional small LLM call	Finds the segments matching a hint (edits, client history)	ms; ~$0.002 and 1–3 s when the LLM step runs

1. Builder

The output models are Pydantic. `DomainCode` is an `Enum` generated from the topic catalog's L1 codes, so Azure enforces it while the model generates. A new domain means a new prompt version.

```
Phase = Literal["opening", "needs_and_situation", "advice", "administration",
"closing"]
Signal = Literal["instruction", "consent_discussion", "complaint",
                 "product_mention", "risk_disclosure", "cancellation"]

class Segment(BaseModel):
    segment_id: str                 # renumbered by the validator
    phase: Phase
    turn_start: TurnId              # "T041"
    turn_end: TurnId
    title: str                      # <= 8 words, in the CA's summary
language
    gist: str                       # <= 40 words, placeholders verbatim
    initiated_by: Literal["CLIENT", "CA", "MIXED"]
    domains: list[DomainCode]
    signals: list[Signal]

class ConversationMap(BaseModel):
    call_purpose: str
    segments: list[Segment]
```

Stage block (frozen, versioned; the transcript is already in the cached prefix):

```
TASK: Build the outline of this call.
- Split the call into 4-20 contiguous segments that together cover every turn
from T001 to T{last_turn}.
- Start a new segment when the subject changes, not when the speaker changes.
- Give each segment one phase. Phases normally run in order but may repeat
(advice -> administration -> advice).
- title: at most 8 words. gist: at most 40 words, factual, placeholders
copied exactly. Write both in {summary_language}.
- domains: every domain below that the segment touches, even briefly. When
unsure, include it.
- signals: only from the closed list in the tool schema.
DOMAINS:
{for each L1 domain: CODE | one-line definition | 3-5 typical cues}
Respond by calling emit_conversation_map.
```

Titles and gists are written in the CA's language on purpose: CAs type their edit hints in that language, so full-text search matches without translation.

2. Validator and repair

`repair_map` makes the map usable even when the model's output is slightly off. It also decides whether a retry is needed.

```
def repair_map(m: ConversationMap, n_turns: int) -> tuple[list[Segment],
list[str]]:
    issues, fixed, cursor = [], [], 1
    for s in sorted(m.segments, key=lambda s: tnum(s.turn_start)):
        a, b = max(tnum(s.turn_start), 1), min(tnum(s.turn_end), n_turns)
        if b < cursor:                                  # fully inside an
earlier segment
            issues.append(f"dropped {s.segment_id}"); continue
        if a > cursor:                                  # gap -> extend the
previous segment
            issues.append(f"gap T{cursor}-T{a - 1}")
            if fixed: fixed[-1].turn_end = tid(a - 1)
            else: a = 1
        a = max(a, cursor)                              # overlap -> cut the
later segment
        fixed.append(s.model_copy(update={"turn_start": tid(a), "turn_end":
tid(b)}))
        cursor = b + 1
    if not fixed:
        raise MapInvalid("no usable segments")
    if cursor <= n_turns:                               # tail gap
        fixed[-1].turn_end = tid(n_turns); issues.append("tail gap")
    fixed = merge_short(fixed, min_turns=2)             # prefer a neighbour
with the same phase
    fixed = cap_count(fixed, max_segments=20)           # merge the smallest
adjacent same-phase pairs
    fixed = check_text(fixed, transcript_placeholders)  # length caps;
unknown placeholder -> blank gist + flag
    return renumber_and_derive_phases(fixed), issues

def domains_to_check(segments, lexicon_hits, cap=8):
    found = {d for s in segments for d in s.domains} | lexicon_hits
    return sorted(found, key=lambda d: (not REGULATORY[d], d))[:cap]   #
regulatory domains first
```

When to retry. Retry once if coverage before repair is below 90%, if the issues exceed max(3, 10% of segments), or if a call of more than 100 turns has fewer than 3 segments. The retry appends the issue list after the stage block, so the cached prefix is still reused.

3. Fallback

If the retry also fails, `fallback_map` builds a deterministic outline and the run is flagged `map_fallback = true`:

- **Segments:** windows of about 25 turns, each boundary moved to the longest pause within ±5 turns (Stage 0 timestamps). Titles read "Part n" and gists are empty.
- **Domains:** lexicon-hit domains plus all regulatory domains, capped at 8, so topic recall does not depend on the failed call.
- **Downstream:** the summary runs without a segment skeleton, and the UI shows the plain outline. **Proposed target:** fallback rate below 1%.

4. Storage and full-text index

Rows go into `conversation_segment` (DDL under *Persisting the conversation structure*). PostgreSQL's built-in full-text search covers the navigator's first step.

- **Use the `simple` configuration.** It does no stemming, so it is safe across DE/FR/IT/EN.
- **Wrap `unaccent` in an immutable function.** `unaccent()` itself is not immutable, so it cannot appear directly in a generated column or index.

```
create extension if not exists unaccent;
create or replace function immutable_unaccent(text) returns text
  language sql immutable parallel safe as $$ select
public.unaccent('public.unaccent', $1) $$;

alter table conversation_segment add column search tsvector
  generated always as (to_tsvector('simple',
      immutable_unaccent(coalesce(title, '') || ' ' || coalesce(gist, ''))))
stored;
create index conversation_segment_search on conversation_segment using gin
(search);

-- step 1 of the navigator: rank one call's segments against a CA hint
select segment_id, ts_rank(search, q) as rank
from conversation_segment, websearch_to_tsquery('simple',
immutable_unaccent(:hint)) q
where run_id = :run_id and search @@ q
order by rank desc limit 5;
```

5. Navigator (`locate_segments`)

The navigator answers "which part of the call is the CA talking about?" in two steps, and the LLM step runs only when full-text search is not decisive.

```
class SegmentPick(BaseModel):
    segment_id: str
    reason: str                                  # one line, for the audit
log

class NavResult(BaseModel):
    picks: list[SegmentPick]                     # 0-3 picks; empty = "not
found"

def locate_segments(run_id: UUID, hint: str, max_picks: int = 3) ->
list[str]:
    hits = db.fts_segments(run_id, hint)                    # SQL above
    if hits and hits[0].rank >= FTS_MIN and (len(hits) == 1 or hits[0].rank
>= 1.5 * hits[1].rank):
        return [hits[0].segment_id]                         # decisive: no
LLM call
    segments = db.segments(run_id)
    outline = render_outline(segments)    # "S04 | needs_and_situation |
T041-T088 | Inheritance ... | gist"
    nav = llm.call_small(tool="pick_segments", model=NavResult,
effort="none",
                         developer=NAVIGATOR_RULES, user=f"{outline}\n\nHINT:
{hint}")
    valid = {s.segment_id for s in segments}
    return [p.segment_id for p in nav.picks if p.segment_id in valid]
[:max_picks]

def segment_context(run_id: UUID, segment_ids: list[str], pad: int = 2,
max_tokens: int = 4000) -> str:
    ranges = merge_ranges(expand(db.turn_ranges(run_id, segment_ids), pad))
    return render_turns(db.turns(run_id, ranges), budget=max_tokens)   #
first-ranked segment kept whole
```

- **The routing call is outside the transcript cache family.** It sends only the outline (~1k tokens) with its own one-tool list, so it costs about $0.002.
- **An empty result is a valid answer.** The UI then asks the CA to pick the section by hand. It never guesses.
- **Every navigation is logged** (hint, FTS ranks, picks, reasons) for evaluation.

6. Navigating across calls (client history, phase 2)

The same two steps work across a client's calls, without embeddings.

1. **SQL pre-filter.** Filter by `client_key`, date range and domains (GIN index), and rank by full-text search. Keep the top ~60 segments: title + gist + date, about 2.5k tokens.

2. **Routing call.** `pick_segments` chooses up to 5 segments across calls.
3. **Answer call.** Load those segments' turns from each call's canonical transcript. The answer cites `call_id` + turn IDs.

Add segment embeddings (`vector(1536)` column, already in the DDL) only if evaluation shows the full-text pre-filter missing relevant segments.

7. Evaluating the map

Check	How	Proposed target
Coverage	Guaranteed by `repair_map`; track repair and fallback rates	Fallback < 1%
Boundary quality	Pk / WindowDiff against gold segmentation on 50 annotated calls	Improves release over release
Domain routing recall	Share of gold topic domains in `domains_to_check`	≥ 0.97
Navigator accuracy	100 sampled edit hints: correct segment among the picks	≥ 0.90
Navigator cost	Share of hints resolved by full-text search alone	Tracked, no target

Persisting the conversation structure

Yes, persist the map as first-class data: one row per segment, about 2 KB per call. It becomes the index for later processing, and re-using it avoids another map call.

Later use	How the stored structure helps
Summary editing	Section → segment → turn range without another LLM call
Incremental reprocessing	A topic-catalog or CIT-catalog change re-runs only the affected adjudications against the stored map. That saves ~$0.05 per call and keeps segment references stable
Client history	A per-client timeline of segments across calls ("what has the client said about the mortgage this year?"): navigate segment titles and gists first and open turns only where needed, the same tree-navigation idea applied across calls

Later use	How the stored structure helps
Analytics	Time spent per domain, share of advice, coverage of required disclosures per desk
Evaluation and audit	Routing recall per domain; why a topic was or wasn't checked
CA UI	Call outline with jump-to-audio

Design rules

- **Turn IDs are the durable anchor; segment IDs are derived.** The canonical transcript never changes, so turn IDs never change. All evidence everywhere is stored as turn IDs. If the map prompt changes and a call is re-mapped, downstream rows re-link through turn ranges.
- **Version the map** with the `map_prompt_version_id` that produced it. Keep superseded maps until their downstream rows are re-linked.
- **Privacy.** Gists can contain client names, so segment rows are client data: encrypted at rest and access-controlled like the transcript. Rows follow the transcript's retention and link to a client only through a pseudonymous `client_key`.
- **Optional, for later:** add a segment embedding (title + gist) once corpus-wide search across many calls is needed. Within one call, navigating the tree is enough.

```
create table conversation_segment (
  run_id uuid, map_prompt_version_id bigint, segment_id text, parent_id text,
level smallint,
  turn_start text, turn_end text, start_ms int, end_ms int,
  title text, gist text, initiated_by text, domains text[], signals text[],
  client_key text, call_date date, embedding vector(1536),   -- nullable,
phase 2; 1,536 dims keeps it indexable
  primary key (run_id, segment_id));
create index on conversation_segment using gin (domains);
create index on conversation_segment (client_key, call_date);
```

Stage 2a: summarization and summary guideline specification

The summary is structured JSON with fixed sections, and every bullet cites turn IDs. Its guidelines are a versioned row in Postgres that the business owns. The UI and CRM render the JSON to text, so edits, translation and audits all work at section level.

Section specification (proposed v1; confirm with business and Compliance)

#	Section code	Content	Rules	Cap
0	`header`	Date, duration, CA, client placeholder, channel	Filled from system metadata, never by the LLM	—
1	`purpose`	Why the call happened and who initiated it	One factual statement	40 words
2	`client_situation`	Changes in the client's situation, goals and constraints	Client-stated facts only	5 bullets
3	`discussion`	One block per substantive map segment	Merge trivial segments; skip small talk	3 bullets per block
4	`advice_given`	Products discussed, CA recommendations with their stated rationale, risk disclosures made, client reaction	Record what was said, never evaluate it; supports FinSA documentation duties (confirm scope with Compliance)	6 bullets
5	`client_instructions`	Instructions and requests from the client	Written independently of CIT detection (see cross-check below)	8 bullets
6	`next_steps`	Action, owner (CA / client / bank unit), due date	Only dates actually stated; never infer one	6 bullets
7	`open_points`	Unresolved questions and pending documents	—	4 bullets
8	`concerns_raised`	Complaints or dissatisfaction the client expressed	Factual; no judgement of tone	3 bullets

Writing rules (injected in the stage block)

1. **Output language** is the CA's configured language (DE/FR/IT/EN), whatever language the call was in. Evidence quotes stay in the original language.

2. **Register:** third person, past tense, neutral and factual ("The client stated…", "The CA recommended…").
3. **Attribution:** attribute every statement to its speaker. Never present a CA proposal as a client decision unless the client explicitly agreed, and cite the turn where they agreed.
4. **Transcript only:** no inferred motives or emotions, and no characterization of the client's personality.
5. **Numbers exact:** amounts, currencies, percentages and dates as stated. If something is ambiguous, write "(unclear in call)".
6. **Client data:** names as stated; placeholders of masked types such as `[DOB_1]` copied exactly. Do not copy IBANs, account or card numbers, phone numbers, email addresses or dates of birth into the summary; the task form holds them ("the son's account").
7. **Data minimization:** leave out sensitive personal details (health, family matters, beliefs) unless they bear directly on an instruction or the advice. If included, keep them minimal and factual.
8. **Final state wins:** when the client changed their mind, record the final position, and mention the change only if it matters.
9. **Bullet format:** at most 30 words and 1–3 cited turn IDs per bullet. Empty sections return `[]`.
10. **Total length** is 150–400 words, scaled to how much happened in the call. This is controlled through the caps above, not `verbosity`, to keep the cache envelope uniform.

Output contract (`emit_summary`, abridged)

```
{
  "language": "de",
  "purpose": {"text": "...", "turns": ["T003"]},
  "sections": [
    {"section_code": "advice_given", "segment_id": "S06",
     "bullets": [{"text": "...", "turns": ["T120", "T124"], "speaker_basis":
"CA"}]}
  ]
}
```

Stage block inputs. Guideline version (~1.5–2.5k tokens) plus the map's segment list (titles, gists, turn ranges, ~1k tokens). Reasoning effort `low`, ~1–1.5k output tokens.

Deterministic checks after the call

- Every cited turn exists and falls within the bullet's segment.
- Section caps are respected.

- Placeholders used (masked types) exist in the transcript, and no IBAN, email address, phone number or date of birth appears in a bullet.
- **Number grounding:** every number or amount in a bullet appears in one of its cited turns, after normalization. Bullets that fail are flagged in the UI.

Cross-check that comes for free. The summary's `client_instructions` and CIT detection run independently and in parallel. At publish time they are matched on overlapping turn IDs:

- **A summary instruction with no matching CIT** is flagged as a possible missed task.
- **A CIT with no matching summary bullet** is flagged too.

This gives a second, independent recall check at almost no extra cost.

Stage 2b–4: CIT detection and parameter extraction

The CIT flow detects once over a compact rendering of the whole task catalog, then extracts once per detected instance using that task's full parameter definitions. Both prompts are generated from the Postgres catalog at run time, so adding a task type or parameter needs no code change, only a catalog version that passes the eval gate.

Step 1: detection (`detect_cits`, reasoning effort medium)

The stage block renders each active task as a compact entry, about 50–80 tokens:

```
EMAIL_WAIVER | Email communication waiver
  def: Client agrees to receive bank correspondence via unencrypted email,
waiving secure-channel protection.
  triggers: "schicken Sie mir das einfach per Mail", "envoyez-moi tout par e-
mail", "I'm fine with normal email"
  NOT: client asks for one document by email without accepting the waiver ->
mentioned_only
  risk: high
```

With 30–60 task types this is 2–5k tokens. Above ~150 task types, filter the entries by the map's domains, the same hierarchical trick used for topics.

What counts as client-induced: the client initiated the request, *or* the CA proposed it and the client explicitly agreed. Candidates whose status is `mentioned_only` or `cancelled` are kept for audit and shown greyed out, but not extracted.

```
{"candidates": [{
  "instance_id": "C1", "task_code": "EMAIL_WAIVER",
  "status": "requested | confirmed | cancelled | mentioned_only",
  "initiated_by": "CLIENT", "client_agreement_turn": "T214",
  "evidence": [{"turn": "T212", "quote": "..."}],
  "confidence": 0.86, "rationale": "Client explicitly accepted email risk
after CA explanation."
}]}
```

The instructions favour recall: when in doubt, emit the candidate with lower confidence. The same task type may appear more than once, for example two separate contract changes.

Step 2: extraction (`extract_cit` × n, reasoning effort low)

The stage block holds:

- the task definition;
- a parameter table rendered from `cit_task_parameter`;
- the candidate's evidence turns as anchors (the full transcript is already in the cached prefix).

```
{"task_code": "EMAIL_WAIVER", "instance_id": "C1", "params": [
  {"param_code": "client_email", "value": {"kind": "email", "value":
"hans.mueller@bluewin.ch"}, "status": "stated",
   "evidence": [{"turn": "T031", "quote": "..."}]},
  {"param_code": "client_consent", "value": "yes", "status": "stated",
   "evidence": [{"turn": "T214", "quote": "..."}]}
]}
```

Extraction rules

- **Respect `source_speaker`.** A client email must be stated or confirmed by the client, and is never the CA's own address.
- **Latest correction wins.** If a conflict is never resolved in the call, set `status = conflicting` and give both pieces of evidence.
- **Tri-state consent.** Consent fields are `yes` / `no` / `not_discussed` and never default to `yes`.
- **Explicit values only.** Client data comes back normalized: emails as lowercase name@domain, IBANs uppercase without spaces, phones as +41..., names and addresses as stated; masked types stay placeholders. Unclear digits or letters mean status unclear, never a guess. Dates are normalized to ISO only when stated explicitly, and amounts always carry their currency.

Step 3: code validation, then verification for high-risk tasks

- **Type checks against the catalog:** IBAN mod-97 checksum, email syntax, phone length, enum membership, tri-state values. Second reading of spoken values: for IBANs, emails and phone numbers the code parses the cited turn itself (digits read out as words, "dot"/"at"/"punkt") and flags any difference from the model's value, which catches misheard digits before a payment goes out with the wrong account.
- **Missing required parameters** set the task to `incomplete`, and the UI prompts the CA for them.
- **`verify_cit` for `risk_tier = high`** (reasoning effort medium) re-checks each parameter against the transcript and returns `agree` / `disagree` with a correction. Disagreements are highlighted for the CA and never auto-resolved.

Example catalog entries (illustrative; your catalog is authoritative)

Task	Parameter	Type	Source speaker	Required	Notes
`EMAIL_WAIVER`	`client_email`	email	CLIENT	Yes	Never the CA's address
`EMAIL_WAIVER`	`scope`	enum: all / specific_documents / unclear	ANY	Yes	
`EMAIL_WAIVER`	`risks_explained`	tri-state	CA	Yes	Evidence = CA's risk explanation
`EMAIL_WAIVER`	`client_consent`	tri-state	CLIENT	Yes	Always CA-confirmed
`DIGITAL_BANKING_CONTRACT`	`action`	enum: create / add_user / change_rights / change_limits / block / terminate	ANY	Yes	One task type with an action enum, not six task types
`DIGITAL_BANKING_CONTRACT`	`authorized_persons`	list of persons	CLIENT	If add_user	
`DIGITAL_BANKING_CONTRACT`	`access_level`	enum: view / transact / full / unclear	ANY	If create or change_rights	
`DIGITAL_BANKING_CONTRACT`	`client_confirmation`	tri-state	CLIENT	Yes	

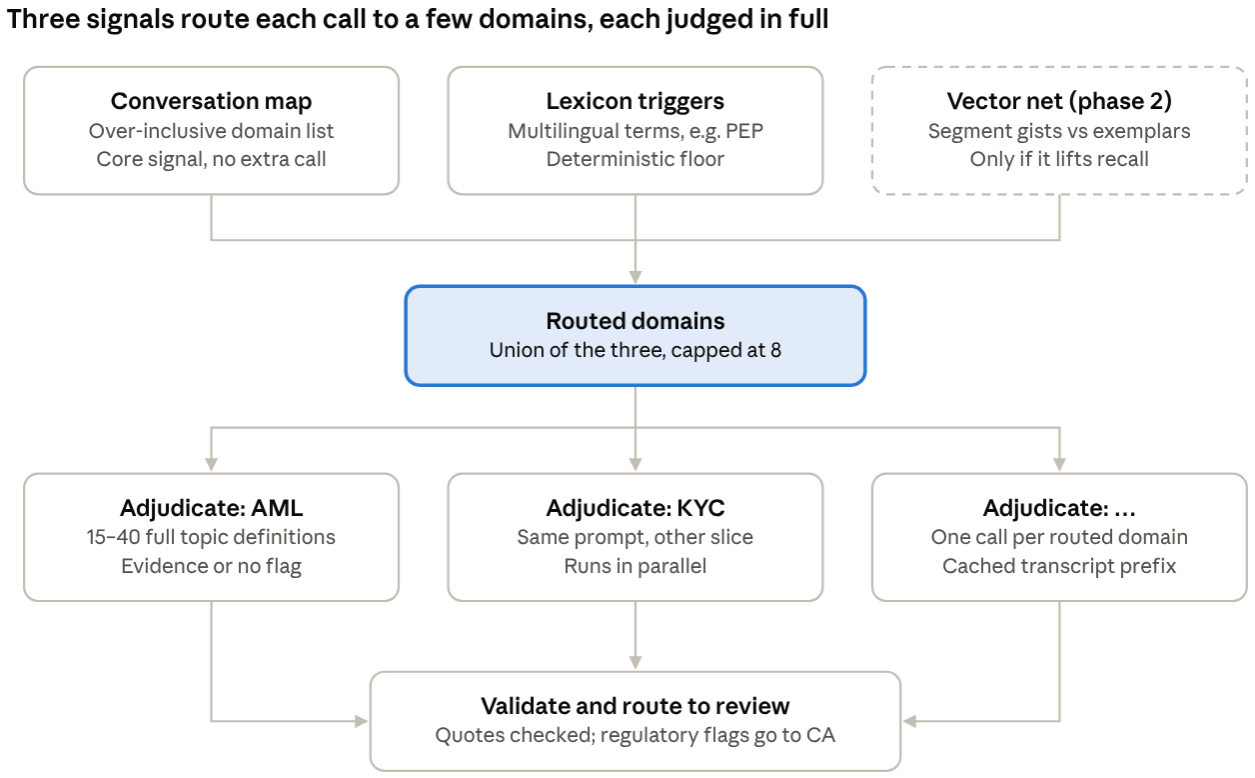
Confusables to encode as `NOT` lines

- **"I can't log in to e-banking"** is a support issue, not contract maintenance.
- **"Send me the factsheet by email"** without accepting email risk is `mentioned_only`, flagged for the CA to confirm.

These boundary cases are where GEPA's feedback is most useful.

Stage 2c: topic flagging over the 350+ topic catalog

Recommendation: hierarchical, vectorless routing with per-domain adjudication, backed by a deterministic lexicon and, optionally, a vector safety net. Settle the final choice in a four-way bake-off on the golden set, with this design as the expected winner.



Topic flagging · 3 routing signals, per-domain adjudication

Routing decides which few domains are worth a full look; adjudication then reads every topic definition in those domains against the whole transcript.

Techniques compared

Approach	How it works	Strengths	Weaknesses	≈ LLM cost per call	Verdict
A. Single shot, full catalog	All 350+ definitions (~20–30k tokens) and the transcript in one call; catalog placed first so it caches	Simplest: one call	Picking from hundreds of labels at once tends to under-recall rare topics and over-flag generic ones; hard to tune per domain	$0.04–0.06	Bake-off baseline

Approach	How it works	Strengths	Weaknesses	≈ LLM cost per call	Verdict
B. Vector match + LLM verify	Embed segment gists and topic exemplars; take the top-k topics per segment; an LLM verifies them	Cheap retrieval; scales to thousands of topics	Similarity is not relevance: implicit cues such as "inheritance from abroad" signal source of wealth (AML) without sharing words; spoken language vs catalog wording; the retriever caps recall	~$0.03	Safety net only
C. Hierarchical, vectorless	The map routes segments to ~20 L1 domains; per-domain LLM adjudication with the full topic definitions	Relevance decided by reasoning; explainable path; small, focused prompts; evaluable per domain; cache-friendly	Depends on routing recall; needs a catalog hierarchy	~$0.06	Core
D. Hybrid (recommended)	C, plus lexicon triggers and an optional vector net feeding the routing	Highest recall; a deterministic floor for regulatory terms	More moving parts	$0.06–0.07	**Build this**

Topic flagging needs what is *relevant*, which takes reasoning over structure; vector search returns what is *similar*. Here the topic tree is the structure, and the conversation map is what gets routed through it. Both are built and stored in-house (see Stage 1).

Topic catalog structure

- **Hierarchy:** L1 domain (~15–25, e.g. KYC, AML, Account Opening, Investment Advice, Credit, Digital Banking, Tax, Complaints) → optional L2 group → L3 topic (350+).
- **Fields per topic:**
  - `definition`
  - `include_criteria` and `exclude_criteria`, with `confusable_with` codes
  - `trigger_terms`, a multilingual lexicon
  - `exemplar_utterances`, synthetic spoken examples in DE/FR/IT/EN
  - the `regulatory` flag and `severity`
  - `allowed_levels`

- **If today's catalog is flat:** build the hierarchy once, offline. An LLM proposes the domain clustering, SMEs review it, and the result is stored as `parent_id` / `ltree` paths.

Runtime flow

1. **Domain routing.** The map validator derives `domains_to_check` from the segment domains, which the map call over-includes by instruction, at no extra cost.
2. **Lexicon booster (deterministic).** Per-topic multilingual terms and patterns, e.g. "PEP", "politisch exponiert", "Herkunft der Gelder", "source of wealth", "US person", "Steuerdomizil". A hit adds the topic's domain and marks that topic `must_consider`.
3. **Vector net (phase 2, only if the bake-off shows a recall gain).**
   - Embed each segment as *title + gist + two key turns*: a contextualized chunk, never raw turns.
   - Match against topic exemplar utterances in pgvector and take the top 5 per segment. Their domains are added to the routing.
   - Embedding models were available as regional Standard deployments in Switzerland North even while GPT models were not (Microsoft Q&A), so this step can stay in Switzerland.
4. **Per-domain adjudication** (`adjudicate_topics` × k, reasoning effort low, fanned out).
   - The stage block holds the domain's full topic definitions (15–40 topics, 2–4k tokens), the segments routed to that domain, and the `must_consider` list.
   - The output (`emit_topic_flags`) is `[{topic_code, level: mentioned | discussed | action_agreed, segment_ids, evidence[], confidence}]`.
   - No evidence means no flag.
5. **Post-processing.** Validate the evidence quotes, de-duplicate topics across domains, and send every flag on a regulatory topic to CA review. Cap at 8 domain calls per transcript; merge the smallest domains into one call if routing returns more.

Other recent techniques considered

Technique	Use here
Contextual chunk embeddings (embed each chunk with its surrounding context)	Yes, in the vector net (segment title + gist), never on raw turns
Synthetic exemplar utterances per topic (HyDE-style)	Yes: bridges the gap between catalog wording and how clients actually talk
LLM listwise reranking	Already covered by per-domain adjudication

Technique	Use here
Distilled classifier trained on CA-confirmed flags	Later (6–12 months): a cheap extra recall net once thousands of labels exist
Agentic multi-step tree search	No: a two-level tree of ~20 domains needs one routing step, not an agent loop

The key diagnostic is routing recall, measured separately from adjudication: was the correct domain routed at all? It tells you whether to fix the map prompt, the lexicon, or the adjudication prompt.

Validation, grounding and CA review

Nothing reaches the CRM or downstream systems without passing deterministic checks and, for anything binding, a CA's confirmation. Every CA action is captured as a label.

`validate_and_publish` checks

1. **Evidence verification.** Every quote must match its cited turn after normalization: case, punctuation and whitespace, with a fuzzy-match threshold of about 0.9. A quote that fails is flagged, never silently dropped.
2. **Catalog type checks** on every CIT parameter, plus the required-parameter rule.
3. **Number grounding** for summary bullets.
4. **Reconciliation** between summary instructions and CIT instances (see the summary section).
5. **Re-hydration of placeholders** (masked types only) happens server-side at display time and on CRM write, never inside an LLM call. Unmasked values are stored as extracted, encrypted at rest.

Review routing

Output	CA action required	Before it goes downstream
CIT with `risk_tier = high` (e.g. email waiver, contract actions)	Explicit confirm per parameter	Yes, blocking
CIT, standard risk	Confirm or edit in the task list	Yes
Topic flag on a regulatory topic (KYC, AML…)	Acknowledge	Yes, for the compliance feed

Output	CA action required	Before it goes downstream
Other topic flags	None (visible, editable)	No
Summary	Approve, or edit and approve	Yes, before the CRM note is written
Reconciliation mismatch	Resolve: add the missed CIT or dismiss	Yes

Feedback capture. Store every accept, edit, reject and "add missed item" event with the run ID and prompt versions. Within weeks, this becomes the largest and freshest labelled dataset for evaluation and GEPA.

Prompt-injection hygiene. `GLOBAL_RULES` states that the transcript is data, never instructions. Forced tool outputs and code validation limit the damage a bad output can do: no LLM call has any side effect beyond writing its own result row.

Predefined-prompt editing of the AI summary

Edits work like this:

- **Synchronous API calls**, not Durable activities.
- **Section-scoped**: an edit changes one section unless the action is whole-summary.
- **Driven by a preset action catalog** in Postgres.
- **Three source modes:** none, map-navigated segments, or the full transcript.
- **Every edit creates a new summary version**, validated the same way as the original.

Action catalog (`summary_edit_action`, v1 proposal)

Code	CA label	Scope	Source mode	Parameters	Effort	Target latency
`SHORTEN`	Make shorter	Section or whole	none	`target_words`	none	1–3 s
`TONE_FORMAL` / `TONE_SIMPLE`	Change tone	Section or whole	none	—	none	1–3 s
`TRANSLATE`	Translate	Whole	none	`language`	none	2–5 s
`CRM_FORMAT`	Convert to CRM note	Whole	none	`template_code`	none	2–4 s
`BULLETS_PROSE`	Bullets ↔ prose	Section	none	—	none	1–3 s

Code	CA label	Scope	Source mode	Parameters	Effort	Target latency
`EXPAND`	Add more detail	Section	segments	—	low	4–8 s
`ADD_POINT`	Add a missing point	Section	segments (routed)	`ca_hint`	low	4–8 s
`CORRECT`	Correct a fact	Section	segments	`ca_note`	low	4–8 s
`REGENERATE`	Regenerate section	Section	full transcript	—	low	8–15 s
`FREE_TEXT`	Custom instruction	Section or whole	routed	text, max 300 characters	low	3–10 s

Labels are stored in DE/FR/IT/EN. New actions are catalog rows, not code changes.

Source modes: the conversation map as vectorless retrieval

- **`none`:** the current section JSON only. A tiny prompt, used for pure style transformations.
- **`segments`:** the section's `segment_id` gives a turn range from the map; send those turns plus 2 on either side, typically 1–4k tokens instead of 12k.
  - For `ADD_POINT` or `FREE_TEXT` with a hint like "the part about the mortgage", first try a deterministic keyword match against segment titles and gists.
  - Failing that, a tiny routing call picks segments from the map (~1k tokens, effort none). This is the Stage 1 navigator (locate_segments), described in the conversation-map implementation.
- **`full`:** the whole canonical transcript, used only for `REGENERATE`.

Call layout

```
[developer] EDIT_RULES (fixed, versioned): edit only the target section; keep
citations;
            new facts only from SOURCE, each with turn IDs; never alter
numbers or placeholders
            unless CORRECT cites SOURCE; output language = summary language
unless TRANSLATE
[user]      CURRENT SECTION(S): JSON
[user]      SOURCE (optional): turns T041-T088
[user]      ACTION: rendered template + parameters
-> forced tool emit_section_patch {section_code, bullets[{text, turns[]}]}
(streamed to the UI)
```

Guardrails

- **No new facts on style-only actions (deterministic).** The numbers, amounts, dates, names, client identifiers, placeholders and catalog product names in the output must be a subset of those in the input. On a violation, retry once, then show the result with a warning.
- **Grounded actions** get the same citation and number-grounding checks as the main pipeline.
- **Version chain.** `summary_version(id, parent_id, action_code, params, actor, prompt_version_id, created_at)`, with a diff view, accept/reject and undo.
- **CA locks.** Sections the CA edited by hand become `ca_locked`, and reprocessing never overwrites them.
- **CIT data is never edited through prompts**, only in the task form.

Cost, caching and optimization

- **No caching on edits.** They usually arrive minutes to hours after processing, when the in-memory cache is gone, so don't design them for caching. Extended 24-hour retention exists for gpt-5.1, but it stores cache state on GPU machines; that is a compliance question, and not worth raising for edits that cost $0.002–0.02 each.
- **Hand-written v1 prompts.** Add GEPA later, with a metric of "accepted without a further manual edit" plus the guardrail pass rate.
- **Track action usage per CA.** If `SHORTEN` is pressed on 30% or more of summaries, tighten the base guidelines rather than leaving it to edits.

DSPy + GEPA optimization approach

Use DSPy with GEPA offline to tune the prompts, and export the winning instructions as frozen, approved prompt versions. DSPy never runs in the production pipeline.

- **Current version.** DSPy is on its 3.x line (3.3.0 shipped in August 2026), and GEPA is its headline optimizer (Klu).
- **Why GEPA fits.** GEPA feeds the scoring function's `feedback` text directly into its reflection prompt; given only a number, it is a much weaker optimizer (DSPy docs).

Modules and scoring functions

Module	Inputs → output	Scoring function	Example feedback text
`ConversationMap`	transcript, domain list → map	Turn coverage + domain routing recall	"Missed AML for S04 (T041–T088): gold label is source of wealth"
`Summarize`	transcript, guidelines, map → summary	Calibrated rubric judge + deterministic checks (citations, caps, number grounding)	"Bullet 3 says CHF 250k; T118 says EUR 250k"
`DetectCITs`	transcript, catalog index, map → candidates	F2 (recall-weighted) per task + status accuracy	"Missed EMAIL_WAIVER at T212; flagged ACCOUNT_CLOSURE but the client only asked about fees"
`ExtractCIT`	transcript, task definition, parameters, anchors → parameters	Field accuracy + evidence validity + status	"client_email took the CA's own address at T030"
`AdjudicateTopics`	transcript, domain topics, segments → flags	Per-topic recall/precision, regulatory recall weighted ×3	"Missed PEP_STATUS: lexicon hit 'politisch exponiert' at T305"

Design rules

1. **Optimize instructions only.** Bootstrapped demos would copy whole 12k-token transcripts into every prompt. Allow short evidence-snippet examples (3–5 turns) only if the eval shows a gain.
2. **Catalog text is an input, not part of the prompt.** GEPA tunes generic instructions that must work across all tasks and topics. When its reflections point at weak catalog wording (e.g. a missing `NOT` line), that goes to SME review and a new catalog version, never straight into production.
3. **Production-parity adapter.** GEPA must score prompts in exactly the layout production serves, or the optimized wording won't transfer.

4. **Models.** The task model is gpt-5.1 at production effort settings. The reflection model is gpt-5.1 at reasoning effort `high`; passing a stronger reflection model is the most common GEPA tuning (DSPy docs).
5. **Data.** Use 200–300 golden calls, stratified by language, call length, task types and topic domains. Split validation-heavy: DSPy guidance often suggests about 20% train and 80% validation, because prompt optimizers overfit small train sets (DSPy GEPA notes). Keep a separate held-out test set that GEPA never sees. The golden set holds real client data: restrict annotator access, set a retention period, and disable DSPy's local response cache or encrypt it.
6. **Budget (estimate).** Start with `auto="light"`. About 1,000–2,000 scoring calls per module at $0.02–0.05 each comes to roughly $50–100 per module run, so a few hundred USD for the full suite.

Sketches (verify hook names against the pinned DSPy 3.3.x)

```
class ProdLayoutAdapter(dspy.JSONAdapter):
    def format(self, signature, demos, inputs):
        return [
            {"role": "developer", "content": GLOBAL_RULES},
            {"role": "user", "content": inputs["transcript"]},
            # signature.instructions is the text GEPA evolves
            {"role": "user", "content": render_stage_block(signature,
inputs)},
        ]

def detect_metric(gold, pred, trace=None, pred_name=None, pred_trace=None):
    missed, false_pos, precision, recall = match_cits(gold, pred,
turn_tolerance=5)
    fb = [f"Missed {m.task} near {m.turn}: {m.why}" for m in missed]
    fb += [f"False {f.task} at {f.turn}" for f in false_pos]
    return dspy.Prediction(score=fbeta(precision, recall, beta=2),
                           feedback="\n".join(fb) or "All tasks correct.")

gepa = dspy.GEPA(metric=detect_metric, auto="light",
                 reflection_lm=dspy.LM("azure/gpt-51",
reasoning_effort="high"))
optimized = gepa.compile(DetectCITs(), trainset=train, valset=val)
```

Promotion path

1. Export the optimized instructions to `prompt_version` (text, hash, module, DSPy version, dataset version, validation and test scores).
2. Four-eyes approval by the model owner and the business owner.

3. Shadow mode for 1–2 weeks: run beside the current prompts, CAs see only the current results, compare.
4. Canary to 10% of CAs, then full rollout. Every run records the `prompt_version_id` it used.

Data model: catalogs, prompts and runs

The tables fall into four groups:

- **Catalogs** (business-owned, versioned)
- **Prompts** (model-owner-owned, approved)
- **Runs and telemetry** (append-only)
- **Results**, including the CA feedback that becomes evaluation labels

The DDL below is abridged; add audit columns (`created_by` , `created_at` , `valid_from` , `valid_to`) throughout.

```
-- Catalogs ----------------------------------------------------------------
create table catalog_version (
  id bigserial primary key, catalog text check (catalog in
('cit','topic','summary_guideline','edit_action')),
  version text, status text check (status in ('draft','approved','retired')),
approved_by text);

create table cit_task (
  task_code text, catalog_version_id bigint references catalog_version,
  name text, definition text, triggers jsonb, non_triggers jsonb,
confusable_with text[],
  domain_code text, risk_tier text check (risk_tier in ('standard','high')),
  requires_ca_confirmation boolean default true, active boolean default true,
  primary key (task_code, catalog_version_id));

create table cit_task_parameter (
  task_code text, catalog_version_id bigint, param_code text, name text,
description text,
  data_type text check (data_type in
('string','email','iban','date','amount','enum','boolean',

'tristate','person_ref','person_ref_list','free_text')),
  enum_values text[], required_when text, source_speaker text check
(source_speaker in ('CLIENT','CA','ANY')),
  extraction_guidance text, validation_regex text, ordinal int,
  primary key (task_code, catalog_version_id, param_code));
```

```
create extension if not exists ltree;
create table topic_node (
  topic_code text, catalog_version_id bigint, path ltree,           -- e.g.
AML.SOURCE_OF_WEALTH
  level smallint, name text, definition text, include_criteria text,
exclude_criteria text,
  confusable_with text[], trigger_terms jsonb, exemplar_utterances jsonb,
  regulatory boolean, severity text, allowed_levels text[], active boolean,
  primary key (topic_code, catalog_version_id));

-- optional phase-2 vector net
create table topic_exemplar_embedding (topic_code text, catalog_version_id
bigint,
  lang text, text text, embedding vector(1536));   -- text-embedding-3-large
with dimensions=1536

create table summary_guideline (catalog_version_id bigint primary key,
sections jsonb, rules text);
create table summary_edit_action (code text, catalog_version_id bigint,
labels jsonb, scope text,
  source_mode text, params_schema jsonb, template text, effort text, primary
key (code, catalog_version_id));

-- Prompts ---------------------------------------------------------------
create table prompt_version (
  id bigserial primary key, module text, text text, sha256 text,
output_tool_schema jsonb,
  model_deployment text, reasoning_effort text, dspy_version text,
dataset_version text,
  val_score numeric, test_score numeric, status text, approved_by text[]);

-- Runs and telemetry ----------------------------------------------------
create table pipeline_run (
  run_id uuid primary key, transcript_id uuid, status text,          --
running|complete|partial|failed
  prompt_set jsonb, catalog_set jsonb, started_at timestamptz, finished_at
timestamptz);

create table llm_call (
  id bigserial primary key, run_id uuid, stage text, item_id text,
prompt_version_id bigint,
  deployment text, reasoning_effort text, prompt_cache_key text,
  input_tokens int, cached_tokens int, output_tokens int, reasoning_tokens
int,
  latency_ms int, cost_usd numeric(10,5), status text, error text,
  unique (run_id, stage, item_id, prompt_version_id));               --
```

```
idempotency key

-- Results ------------------------------------------------------------
create table conversation_map (run_id uuid primary key, map jsonb);
create table summary_version (id bigserial primary key, run_id uuid,
parent_id bigint,
  action_code text, action_params jsonb, actor text, content jsonb,
locked_sections text[],
  prompt_version_id bigint, created_at timestamptz);
create table cit_instance (id bigserial primary key, run_id uuid, instance_id
text, task_code text,
  status text, initiated_by text, confidence numeric, evidence jsonb,
review_state text);
create table cit_param_value (cit_instance_id bigint, param_code text, value
text, status text,
  evidence jsonb, validation jsonb, verifier_verdict text, ca_value text,
ca_action text);
create table topic_flag (id bigserial primary key, run_id uuid, topic_code
text, level text,
  segment_ids text[], evidence jsonb, confidence numeric, routed_by text[],
-- map|lexicon|vector
  review_state text, ca_action text);
```

The `routed_by` column on `topic_flag` and the `ca_*` columns are what make routing recall and CA corrections measurable without extra tooling.

PostgreSQL feasibility notes (Azure Database for PostgreSQL Flexible Server)

- **Allow-list the extensions** (`ltree`, `vector`, `unaccent`) in the `azure.extensions` server parameter before running `create extension`.
- **Keep embeddings indexable.** pgvector's HNSW and IVFFlat indexes support up to 2,000 dimensions for the `vector` type. Request 1,536-dimension embeddings through the `dimensions` parameter of text-embedding-3-large, or store full-size vectors as `halfvec`.

Implementation details

The production code is plain Python: the OpenAI SDK against Azure's v1 endpoint, Pydantic v2 for every output contract, Jinja2 for deterministic rendering, and psycopg 3 for Postgres. DSPy lives only in the offline `optimize/` package.

Repository layout

```
call-intel/
├─ function_app/              # Durable Functions: orchestrator + activities
|  ├─ orchestrator.py
|  └─ activities/             # preprocess, map, summarize, detect, extract,
topics, verify, publish
├─ api/                       # FastAPI: CA review + synchronous summary
edits
├─ core/
|  ├─ llm/client.py           # Azure OpenAI wrapper: forced tools, retries,
usage + prefix-hash logging
|  ├─ llm/tools.py            # Pydantic output models -> Azure-compatible
strict tool schemas
|  ├─ prompts/render.py       # deterministic Jinja2 rendering, prefix
hashing
|  ├─ catalogs/               # CIT / topic / guideline loaders and renderers
(cached per version)
|  ├─ pseudo/                 # identifier detection, configurable masking,
egress guard
|  ├─ validate/               # evidence, number grounding, type checks,
reconciliation
|  └─ store/                  # Postgres repositories, result store
(idempotency)
├─ optimize/                  # DSPy programs, prod-layout adapter, metrics,
GEPA runners (offline)
├─ eval/                      # golden set, scorers, regression gate
└─ tests/
```

LLM client wrapper

Use the OpenAI Python SDK against the Azure v1 endpoint with Entra ID tokens, as in Microsoft's examples. Azure requires `parallel_tool_calls=False` when using structured outputs with function calling (Microsoft Learn).

```
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import OpenAI

token = get_bearer_token_provider(DefaultAzureCredential(),
"https://ai.azure.com/.default")
client = OpenAI(base_url=f"https://{RESOURCE}.openai.azure.com/openai/v1/",
                api_key=token, max_retries=0, timeout=120)   # retries
handled in with_backoff

def call_stage(stage, prefix_msgs, stage_block, cache_key):
    resp = with_backoff(lambda: client.chat.completions.create(
        model=DEPLOYMENT,                                    # pinned gpt-5.1
deployment
        messages=prefix_msgs + [{"role": "user", "content": stage_block}],
        tools=ALL_OUTPUT_TOOLS,                              # same list,
same order, every call
        tool_choice={"type": "function", "function": {"name":
stage.tool_name}},
        parallel_tool_calls=False,
        reasoning_effort=stage.effort,
        max_completion_tokens=stage.max_out,
        prompt_cache_key=cache_key))                         # transcript_id
    args = resp.choices[0].message.tool_calls[0].function.arguments
    return args, resp.usage
```

Pydantic structured outputs and regex formats (IBAN, email)

Pydantic can express the formats, but Azure will not enforce them while the model generates. Azure's structured outputs list `pattern` and `format` as unsupported string keywords, alongside `minLength`, `maxLength` and the numeric and array bounds (Microsoft Learn, updated August 2026). OpenAI's own API now accepts `pattern` and `format` (Developers Digest), so re-check Azure with each release. Keep the Pydantic validators either way.

Three layers of format enforcement

Layer	Enforced by	What it covers
While generating	Azure strict schema	Types, required fields, `enum`s, `anyOf` unions. Put every closed set here: statuses, tri-states, value kinds, action enums

Layer	Enforced by	What it covers
When parsing	Pydantic	Turn IDs and ISO dates via `StringConstraints(pattern=…)`. On failure: one repair retry with the validation error appended, then `invalid` for CA review
After parsing	`check_value` in Python, per masking mode	Masked type: must be a known placeholder (a raw value is a leak). Unmasked type: IBAN mod-97 checksum, email syntax, phone length, and a second reading of the cited turn

The second reading catches misheard digits. When a client reads an IBAN out digit by digit, the model has to turn words into digits. The code parses the cited turn itself (digit words in DE/FR/IT/EN, "dot"/"at"/"punkt" in emails) and compares. A checksum-valid IBAN that differs from what was said is flagged as well as an invalid one, so this tests the value itself, which a placeholder regex never could.

```
from typing import Annotated, Literal, Union
import openai
from pydantic import BaseModel, Field, StringConstraints

TurnId  = Annotated[str, StringConstraints(pattern=r"^T\d{3,4}$")]
IsoDate = Annotated[str, StringConstraints(pattern=r"^\d{4}-\d{2}-\d{2}$")]
# Client-data fields: a placeholder if the type is masked, else the
normalized value -> checked in check_value()
IbanStr   = Annotated[str, Field(description="Placeholder like [IBAN_1] if
shown so; else uppercase, no spaces")]
EmailStr  = Annotated[str, Field(description="Placeholder like [EMAIL_1] if
shown so; else name@domain, lowercase")]
PersonStr = Annotated[str, Field(description="Placeholder like [PERSON_2] if
shown so; else the name as stated")]

class IbanValue(BaseModel):    kind: Literal["iban"];     value: IbanStr
class EmailValue(BaseModel):   kind: Literal["email"];    value: EmailStr
class PersonsValue(BaseModel): kind: Literal["persons"];  value:
list[PersonStr]
class DateValue(BaseModel):    kind: Literal["date"];     value: IsoDate
class TriValue(BaseModel):     kind: Literal["tristate"]; value:
Literal["yes", "no", "not_discussed"]
class EnumValue(BaseModel):    kind: Literal["enum"];     value: str   #
checked against the catalog in code
ParamValue = Union[IbanValue, EmailValue, PersonsValue, DateValue, TriValue,
```

```
EnumValue]   # "anyOf"

class Evidence(BaseModel):
    turn: TurnId
    quote: str

class ParamResult(BaseModel):
    param_code: str
    status: Literal["stated", "not_stated", "conflicting", "unclear"]
    value: ParamValue | None
    evidence: list[Evidence]

AZURE_UNSUPPORTED = {"pattern", "format", "minLength", "maxLength",
"minimum", "maximum",
                     "multipleOf", "minItems", "maxItems", "uniqueItems"}

def strip(node):
    if isinstance(node, dict):
        node = {k: strip(v) for k, v in node.items() if k not in
AZURE_UNSUPPORTED}
        if "const" in node:                    # Literal["iban"] -> single-
value enum, the form Azure documents
            node.setdefault("enum", [node.pop("const")])
        return node
    return [strip(v) for v in node] if isinstance(node, list) else node

def azure_tool(model: type[BaseModel], name: str) -> dict:
    tool = openai.pydantic_function_tool(model, name=name)
    tool["function"]["parameters"] = strip(tool["function"]["parameters"])
    return tool

def check_value(value, evidence, turns, pmap, mask) -> list[str]:
    etype = KIND_ETYPE[value["kind"]]                  # iban -> IBAN, email
-> EMAIL, person -> PERSON ...
    v = value["value"]
    if etype in mask:                                  # placeholder expected
        return [] if v in pmap.tokens else [f"raw or unknown {etype} value -
possible leak"]
    issues = []
    if etype == "IBAN" and not iban_valid(v):
        issues.append("IBAN checksum fails - possibly misheard digits")
    seen = {norm(etype, s.canonical) for e in evidence
            for s in pattern_spans(turns[e["turn"]].text) if s.etype ==
etype}
```

```
    if seen and norm(etype, v) not in seen:            # second,
deterministic reading of the spoken value
        issues.append(f"differs from the cited turn: {sorted(seen)[0]}")
    return issues
```

Keep schemas within Azure's limits: at most 100 object properties in total and five levels of nesting (Microsoft Learn). The envelope above uses four levels.

Deterministic rendering

- **Templates.** Jinja2 with `StrictUndefined`, and catalog entries always sorted by code.
- **Nothing volatile in the prefix:** no timestamps, run IDs or UUIDs in tools, developer message or transcript.
- **Build the tool list once.** `ALL_OUTPUT_TOOLS` is built at startup from the Pydantic models in a fixed order and frozen. A change to any output model is a new prompt version.
- **Hash what is sent.** `prefix_sha256` is computed over the exact serialized tools + developer + transcript messages.
- **Snapshot test.** The same inputs must render to the same bytes; any diff fails CI unless the renderer version is bumped.

Activity template

```
@app.activity_trigger(input_name="job")
def extract_cit(job: dict) -> dict:
    key = ResultKey(job["run_id"], "extract_cit", job["candidate_id"],
PV["extract_cit"].id)
    if (hit := store.get_result(key)):                  # result store =
idempotency
        return {"status": hit.status}
    t = store.load_canonical_transcript(job["run_id"])
    prefix = render.prefix(t)                           # tools +
GLOBAL_RULES + transcript
    block = render.extract_block(store.candidate(job["candidate_id"]),
catalogs.cit(t.cit_catalog_version))
    egress_guard(prefix, block)                         # raw value of a
masked type -> block + alert
    try:
        args, usage = llm.call_stage(STAGES["extract_cit"], prefix, block,
cache_key=t.transcript_id)
        result = validate.extraction(args, t, catalogs)  # Pydantic + catalog
types + evidence
        store.save_result(key, result, usage)
        return {"status": "ok"}
    except LlmError as e:
        store.save_failure(key, e)
        return {"status": "failed"}                     # orchestrator
publishes run as partial
```

Schema note. Add `prefix_sha256 text` and `created_at timestamptz default now()` to `llm_call` for the cache diagnostics above.

Libraries

Clear all versions and licenses through the bank's open-source approval process.

Purpose	Library
Azure OpenAI client and auth	`openai` (Python SDK, v1 endpoint), `azure-identity`
Orchestration	`azure-functions`, `azure-functions-durable`
Output models and validation	`pydantic` v2
Prompt rendering	`Jinja2`
Database	`psycopg` 3; `pgvector` in phase 2

Purpose	Library
Identifier detection and masking	`presidio-analyzer`, `spaCy` DE/FR/IT/EN models, `phonenumbers`
Real-value validation	`schwifty` (IBAN), `email-validator`
Review and edit API	`FastAPI`
Telemetry	OpenTelemetry + Azure Monitor exporter
Offline optimization	`dspy` 3.3.x (pinned)

Tests that matter most

1. **Prefix identity.** The same transcript, rendered for every stage, gives an identical prefix hash.
2. **Schema contract.** Each tool schema is accepted by the Azure deployment, run as a nightly smoke call.
3. **Client-data handling.** Detection recall per type on the annotated set (gate on masked types), value-normalisation tests with spoken-form fixtures (IBANs, emails, phones), and egress-guard tests per masking mode.
4. **Validators.** Placeholder regexes, catalog enums, evidence matching and number grounding.
5. **Golden-set regression.** Runs on every prompt, catalog, model or library change.
6. **Resilience.** Simulated 429s and timeouts: retries, partial runs and idempotent replays.

Cost and latency model

The full pipeline costs about **$0.22 per call with caching** and about $0.37 without. At ~11,000 calls a month (500 a day × 22 business days) that is roughly **$2.4k vs $4.1k a month**, so caching saves about 40%.

Prices assumed: Azure Data Zone gpt-5.1 at ~$1.375 per 1M input tokens, $11 per 1M output, and $0.1375 per 1M cached input (cloudprice.net). Reasoning tokens are billed as output.

Per-call LLM cost (estimate, 45-minute call, shared prefix ≈ 15k tokens: transcript + rules + tool definitions)

Stage	Calls	Input tokens (cached)	Output incl. reasoning	≈ cost
1 Map (warm-up)	1	16.5k (0)	2.5k	$0.050

Stage	Calls	Input tokens (cached)	Output incl. reasoning	≈ cost
2a Summary	1	18k (15k)	2k	$0.028
2b CIT detection (medium)	1	20k (15k)	2.5k	$0.036
2c Topic adjudication	4 avg	18.5k (15k) each	0.8k each	$0.063
3 CIT extraction	2 avg	16.5k (15k) each	1k each	$0.030
4 High-risk verification	0.3 avg	17k (15k)	3k	$0.011
Total	**≈ 9.3**			**≈ $0.22**

What moves the number

- **Output and reasoning tokens are about 70% of the cost.** Effort settings and output caps matter more than input tweaks.
- **Cross-stage caching depends on the uniform envelope** (see the prompt layout section). With caching only within each stage's fan-out, expect about $0.26–0.30 per call.
- **The number of topic domains routed** is the main variable. Each extra domain call costs about $0.016.
- **Summary edits** cost $0.002–0.02 each, and regenerating a section about $0.04. Even at two edits for 30% of summaries, that is under $100 a month.
- **GEPA optimization** costs a few hundred USD per full cycle across all modules.

Latency (p50 estimates; validate in the spike)

Step	Estimate
Preprocessing and identifier detection	2–5 s
Conversation map	15–25 s
Stage 2 fan-out (the slowest member, usually detection)	20–35 s
CIT extraction fan-out	8–15 s
High-risk verification (when triggered)	10–20 s

Step	Estimate
Validation and publish + Durable scheduling overhead	2–5 s
End to end	**≈ 50–90 s**

Evaluation, rollout and governance

One golden set and one regression gate govern every change: prompt, catalog, model version or DSPy version. Nothing reaches CAs without passing it.

Golden set. 200–300 calls (real client data, access-restricted), each annotated by two people (CAs or ops) with disagreements adjudicated. Each call carries:

- a reference summary with rubric ratings;
- CIT instances and parameters, with their turns;
- topic flags with levels;
- the gold routing domains.

Top it up monthly from CA corrections in production.

Metrics and proposed gates (calibrate after the first baseline run)

Stage	Primary metric	Proposed gate
CIT detection	Recall on high-risk tasks; F2 overall	⩾ 0.95; ⩾ 0.85
CIT extraction	Field accuracy, normalized	⩾ 0.95 on high-risk parameters; ⩾ 0.90 on others
Topic routing	Domain recall (correct domain routed)	⩾ 0.97
Topic flags	Macro-recall on regulatory topics; precision overall	⩾ 0.90; ⩾ 0.75
Summary	Calibrated judge rubric (factuality, completeness, attribution, guideline compliance); unsupported numbers	Judge–CA agreement ⩾ 0.8; zero unsupported numbers
Evidence	Quote match rate	⩾ 0.98
Edits	Accepted without further manual rework	⩾ 0.80

Stage	Primary metric	Proposed gate
Client-data values	Exact match of extracted IBANs, emails and phone numbers after normalisation; misheard rate (share flagged by the second reading)	⩾ 0.99 exact match; misheard rate tracked per language

Online monitoring. Watch these, per task, topic and section where they apply:

- CA override rates;
- missed-item additions;
- reconciliation mismatches;
- cache hit rate;
- cost per call;
- latency (p50/p95);
- partial-run rate;
- quality split by call language.

Alert when an override rate jumps more than 2× its 4-week baseline.

Rollout

1. Shadow mode on live traffic, with results hidden from CAs.
2. Pilot with ~15 CAs across languages and desks.
3. All 150 CAs, with weekly review of override analytics for the first month.

Governance

- **Pin model versions.** Pin `gpt-5.1` and turn off auto-upgrade. Before any model migration, re-run the golden set and a GEPA cycle on the successor model.
- **Four-eyes approval** for prompt versions (model owner) and catalog versions (business owner).
- **Full lineage on every output:** `run_id`, prompt versions, catalog versions, deployment and token usage.
- **Model inventory entry.** Register the pipeline in the model inventory, with documentation of intended use, limitations, the evaluation results, and the human review controls described above.

Risks, open decisions and next steps

The design has three load-bearing assumptions, each testable within the first two weeks: cross-stage caching on Azure, diarization quality, and a usable topic hierarchy.

Risk	Impact	Mitigation
Cross-stage cache hits lower than expected on Azure Data Zone GPT-5.1	+$1.7k/month at most; no quality impact	Day-one spike; if hits are poor, keep the quality-first layout
Weak diarization (CA and client mixed up)	Wrong "client-induced" decisions	Quality flags; map call infers roles; mandatory CA confirmation
Topic catalog has no usable hierarchy	Routing approach blocked	One-off LLM-assisted clustering with SME review (~1–2 weeks)
Generic extraction schema allows type drift	Malformed parameter values	Strict code validation against the catalog; `incomplete` status prompts the CA
GEPA overfits a small golden set	Prompts regress on live traffic	Validation-heavy split; held-out test gate; shadow mode
DSPy adapter API changes between versions	Optimization loop breaks	Pin DSPy 3.3.x; adapter covered by a unit test of the rendered layout
GPT-5.1 retirement	Forced migration	Golden set + GEPA re-run on the successor model before switching
Model misreads a spoken identifier (IBAN digit, email)	Wrong account or contact data in a task	Second deterministic reading of the cited turn; IBAN checksum; CA confirmation of high-risk tasks
Client data lands in logs, caches or fixtures	Client data outside controlled stores	Telemetry logs counts and hashes only; result store, segments and fixtures encrypted with the transcript's retention

Open decisions

1. Final summary section list and caps (business and Compliance), including the FinSA documentation scope.
2. The CA output language rule (CA profile vs client language).
3. Which CIT task types are `risk_tier = high`.

4. Whether the phase-2 vector net is built, based on the bake-off result.
5. Which identifier types to mask (proposed: date of birth, AHV and card numbers).

Next steps

- [ ] Cache spike: 20 transcripts, uniform envelope vs per-stage schemas, measuring `cached_tokens` per stage
- [ ] Golden set v0: 50 calls annotated, enough for a first baseline
- [ ] Catalog readiness: CIT task and parameter rows, and the topic hierarchy (L1 domains)
- [ ] Topic bake-off: approaches A, C and D on golden set v0
- [ ] Production-parity DSPy adapter and the scoring functions for all five modules
- [ ] First GEPA cycle on `DetectCITs` and `AdjudicateTopics`
- [ ] Edit-action catalog v1 and the deterministic no-new-facts check

Sources

- OpenAI — Prompt caching guide (prefix-affecting settings, cache key routing, retention, regional boundaries)
- Microsoft Learn — Azure OpenAI prompt caching
- Microsoft Learn — Durable Functions fan-out/fan-in
- DSPy — GEPA in depth
- Klu — DSPy overview (versions, GEPA)
- DSPy GEPA optimizer notes (dataset splits)
- Microsoft Q&A — GPT deployments in Switzerland North
- cloudprice.net — GPT-5.1 Azure pricing
- Microsoft Learn — Azure OpenAI structured outputs (unsupported keywords, schema limits, `parallel_tool_calls`)
- Microsoft Learn — prompt caching, `cached_tokens` reporting
- Microsoft Q&A — API Management token metrics and cached tokens
- Developers Digest — structured outputs across APIs, 2026
- Microsoft Presidio